



**LAPORAN PRAKTIKUM TUGAS 1 : RESTFUL API Murni dengan
Express.js**

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Putri Angel Li

NIM : 2428240091

Nomor Topik : 25

Nama Topik : Telekomunikasi: Paket Data

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Putri Angel Li
NPM	2428240091
Kelas	SI5B
Pertemuan Ke-	3,4
Topik / Judul Praktikum	Laporan Praktikum Tugas 1: RESTful API Murni dengan Express.js
Tanggal Pelaksanaan	Kamis, 24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).

1. Mampu membuat project Express.js
2. Mampu membuat routing RESTful
3. Mampu menerima request
4. Mampu mengirimkan response murni dalam format JSON menggunakan Postman
5. Mampu mengimplementasikan RESTful API secara utuh (routing, request, response JSON)
6. Mampu mendeploy menggunakan Vercel

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 2 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

1. REST dan Resource

REST (Representational State Transfer) merupakan gaya arsitektur yang digunakan untuk membangun layanan web dengan memanfaatkan protokol HTTP.

REST memungkinkan aplikasi client berkomunikasi dengan server melalui endpoint yang mewakili suatu sumber daya atau resource. Resource dapat berupa data pengguna, produk, mahasiswa, buku, atau objek lainnya.

Dalam REST API, setiap resource biasanya memiliki URL atau endpoint tertentu. Sebagai contoh, `/data-plans` digunakan untuk mengakses kumpulan data telekomunikasi, sedangkan `/data-plans/1` digunakan untuk mengakses data-plans dengan ID 1. REST juga menggunakan method HTTP untuk menentukan operasi yang dilakukan terhadap resource tersebut.

2. Method HTTP

HTTP menyediakan beberapa method yang umum digunakan dalam REST API, yaitu:

- a) GET digunakan untuk mengambil atau membaca data dari server. Contohnya GET `/data-plans` untuk mengambil seluruh data data-plans.
- b) POST digunakan untuk membuat atau menambahkan data baru. Contohnya POST `/data-plans` untuk menambahkan data-plans.
- c) PUT digunakan untuk memperbarui data yang sudah ada secara keseluruhan. Contohnya PUT `/data-plans/1` untuk memperbarui data data-plans dengan ID 1.
- d) DELETE digunakan untuk menghapus data. Contohnya DELETE `/data-plans/1` untuk menghapus data-plans dengan ID 1.

Penggunaan method tersebut membuat API memiliki pola komunikasi yang jelas antara client dan server.

3. HTTP Status Code

HTTP status code merupakan kode yang dikirimkan server sebagai informasi mengenai hasil suatu request. Status code terdiri dari beberapa kelompok, di antaranya:

- a) 2xx (Success) menunjukkan request berhasil. Contohnya 200 OK untuk request yang berhasil dan 201 Created untuk data yang berhasil dibuat.
- b) 4xx (Client Error) menunjukkan terdapat kesalahan pada request dari client. Contohnya 400 Bad Request ketika data yang dikirim tidak valid dan 404 Not Found ketika resource yang diminta tidak ditemukan.
- c) 5xx (Server Error) menunjukkan terjadi kesalahan pada sisi server. Contohnya 500 Internal Server Error.

Status code membantu client mengetahui apakah request berhasil atau mengalami kesalahan sehingga dapat menentukan tindakan selanjutnya.

4. Format JSON

JSON (JavaScript Object Notation) merupakan format pertukaran data yang banyak digunakan dalam REST API. JSON memiliki struktur sederhana yang terdiri dari pasangan key-value dan dapat digunakan untuk merepresentasikan objek maupun kumpulan data.

Contoh response JSON:

```
{
  namaPaket: 'Internet Bulanan 25GB',
  operator: 'Nusanet',
  kuotaGb: 25,
  masaAktifHari: 30,
  harga: 85000
},
```

JSON banyak digunakan karena formatnya ringan, mudah dibaca manusia, dan dapat diproses oleh berbagai bahasa pemrograman. Dalam REST API, JSON umumnya digunakan sebagai format data pada request maupun response.

5. Express.js

Express.js merupakan framework web untuk Node.js yang digunakan untuk membangun aplikasi web dan REST API. Express menyediakan fitur untuk menangani routing, request, response, middleware, serta berbagai kebutuhan dalam pengembangan aplikasi server.

Dalam pembuatan REST API, Express dapat digunakan untuk menentukan endpoint dan method HTTP yang digunakan. Contohnya:

```
app.get('/data-plans', (req, res) => {
  res.json({ message: 'Data plans' });
});
```

Kode tersebut membuat endpoint GET /data-plans yang memberikan response dalam format JSON. Dengan struktur yang sederhana dan fleksibel, Express.js banyak digunakan untuk membangun server dan REST API berbasis Node.js.

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Versi yang digunakan adalah versi 24, digunakan sebagai runtime environment untuk menjalankan JavaScript pada sisi server.
2	Visual Studio Code	Digunakan sebagai code editor untuk membuat dan mengelola project serta kode program.
3	Express.js	Express yang digunakan adalah express@5.2.1 . Digunakan sebagai runtime environment untuk menjalankan JavaScript pada sisi server.
4	Postman	Digunakan untuk melakukan pengujian RESTful API dengan mengirim request dan melihat response dari server.
5.	Restful API	Digunakan sebagai konsep/arsitektur komunikasi antara client dan server melalui HTTP.
6.	Akun Github	https://github.com/Putriangelli75/tugas1-restful-2428240091
7.	Akun Vercel	https://tugas1-restful-2428240091.vercel.app/

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. Install dan pasang NVM jika belum ada, setelah install buka command prompt dengan menuliskan `nvm install <versi>`, tunggu sampai selesai. Setelah itu ketik `nvm list` dan lihat apakah sudah terinstall.
2. Lalu setelah itu, ketik `D:` lalu ketik `mkdir tugas1-restful-2428240091` untuk membuat folder, dan terakhir `CD tugas1-restful-2428240091`.
3. Ketikkan `npm init -y` di command prompt, lalu setelah itu ketik `npm install express`.

4. Setelah file dibuka di Visual Studio Code ubah bagian package.json dan dibagian main ubah menjadi app.js, lalu buat file baru dengan nama app.js
5. Ketik npm install --save-dev nodemon pada terminal, nodemon ini membantu agar server restart otomatis.

6. Tambahkan bagian scripts pada package.json:

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js"  
}
```

7. Test apakah npm run dev berjalan di terminal, lalu buka file app.js

8. Tambahkan text pada app.js :

```
const express = require('express');  
const app = express();  
const PORT = 3000;  
  
// GET / -> memastikan server berjalan  
app.get('/', (req, res) => {  
  res.json({  
    nama: "Putri Angel Li",  
    nim: "2428240091",  
    topik: 25,  
    endpoints: [  
      "GET /data-plans",  
      "GET /data-plans/:id",  
      "POST /data-plans",  
      "PUT /data-plans/:id",  
      "DELETE /data-plans/:id"  
    ]  
  });  
});  
  
app.listen(PORT, () => {  
  console.log(`Server berjalan di http://localhost:${PORT}`);  
});
```

9. Tambahkan di bawah `const PORT = 3000`; sebelum route apa pun:

```
// Middleware agar req.body (JSON) dapat dibaca
app.use(express.json());

// Data sementara (disimpan di memori, hilang saat server restart)
let dataplans = [
  { id: 1, namaPaket: 'Internet Bulanan 25GB', operator: 'Nusanet',
    kuotaGb: 25, masaAktifHari: 30, harga: 85000 },
  { id: 2, namaPaket: 'Internet Bulanan 10GB', operator: 'Telkomsel',
    kuotaGb: 10, masaAktifHari: 30, harga: 50000 },
  { id: 3, namaPaket: 'Internet Bulanan 5GB', operator: 'Axis', kuotaGb: 5,
    masaAktifHari: 30, harga: 30000 }
];
let nextId = 4;
```

10. Tambahkan sebelum `app.listen(...)`:

```
// GET / -> memastikan server berjalan
app.get('/', (req, res) => {
  res.json({
    nama: "Putri Angel Li",
    nim: "2428240091",
    topik: 25,
    endpoints: [
      "GET /data-plans",
      "GET /data-plans/:id",
      "POST /data-plans",
      "PUT /data-plans/:id",
      "DELETE /data-plans/:id"
    ]
  });
});
```

11. Tambahkan `GET/data-plans` dibawah `/GET` :

```
// GET /data-plans -> seluruh data,
// bisa difilter: /data-plans?operator=Nusanet
app.get('/data-plans', (req, res) => {
  const { operator } = req.query;
```

```

    if (operator) {
        const hasil = dataplans.filter((d) => d.operator === operator);
        return res.json(hasil);
    }

    res.json(dataplans);
});

```

12. Tambahkan GET/data-plans/:id (menampilkan satu data berdasarkan id) :

```

/ GET /data-plans/1 -> satu data berdasarkan id
app.get('/data-plans/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const data = dataplans.find((d) => d.id === id);

    if (!data) return res.status(404).json({
        "status": "error",
        "message": `Data dengan id ${id} tidak ditemukan`,
        "data": null
    });
    res.json(data);
});

```

13. Tambahkan POST/data-plans (menambahkan data baru) :

```

// POST /data-plans -> tambah data baru
// Body JSON: { "namaPaket": "Internet Bulanan 50GB", "operator":
"Indosat", "kuotaGb": 50, "masaAktifHari": 30, "harga": 120000 }
app.post('/data-plans', (req, res) => {
    const { namaPaket, operator, kuotaGb, masaAktifHari, harga } =
req.body;

    // validasi : field wajib kosong -> 400
    if (!namaPaket || !operator || !kuotaGb || !masaAktifHari || !harga) {
        return res.status(400).json({
            "status": "error",
            "message": 'namaPaket, operator, kuotaGb, masaAktifHari, dan
harga wajib diisi',
            "data": null,
        });
    }
}

```

```

const baru = { id: nextId++, namaPaket, operator, kuotaGb,
masaAktifHari, harga };
dataplan.push(baru);

//berhasil -> 201 + data yang baru dibuat
res.status(201).json({
  "status": "success",
  "message": "Data berhasil ditambahkan",
  "data": baru,
});
});

```

14. Tambahkan PUT/data-plans/:id (mengubah satu data berdasarkan id) :

```

// PUT /data-plans/4 -> ubah data
app.put('/data-plans/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = dataplan.findIndex((d) => d.id === id);

  if (index === -1) {
    return res.status(404).json({
      "status": "error",
      "message": `Data dengan id ${id} tidak ditemukan`,
      "data": null
    });
  }

  dataplan[index] = { ...dataplan[index], ...req.body, id };

  res.status(200).json({
    status: "success",
    message: "Data berhasil diperbarui",
    data: dataplan[index]
  });
});

```

15. Tambahkan DELETE/data-plans/:id (menghapus satu data berdasarkan id) :

```

app.delete('/data-plans/:id', (req, res) => {

```

```

const id = parseInt(req.params.id);
const index = dataplans.findIndex((d) => d.id === id);

if (index === -1) {
  return res.status(404).json({
    "status": "error",
    "message": `Data dengan id ${id} tidak ditemukan`,
    "data": null
  });
}

dataplans.splice(index, 1);
res.status(200).json({
  "status": "success",
  "message": `Data data-plans dengan id ${id} berhasil dihapus`,
  "data": null
});
});

```

16. Ubah `const PORT = 3000;` menjadi `const PORT = process.env.PORT || 3000;`

17. Ubah `app.listen(PORT, () => {`
`console.log(`Server berjalan di http://localhost:${PORT}`);`
`});`

Menjadi `if (process.env.NODE_ENV !== 'production') {`
`app.listen(PORT, () => {`
`console.log(`Server berjalan di http://localhost:${PORT}`);`
`});`
`}`

Dan tambahkan `module.export = app;` dibawahnya

18. Jalankan ulang `npm run dev` di terminal, klik servernya: <http://localhost:3000/> ;
 maka yang tampil adalah data GET/ :

```

{
  "nama": "Putri Angel Li",
  "nim": "2428240091",
  "topik": 25,
  "endpoints": [

```

```

    "GET /data-plans",
    "GET /data-plans/:id",
    "POST /data-plans",
    "PUT /data-plans/:id",
    "DELETE /data-plans/:id"
  ]
}

```

Artinya server berhasil berjalan

19. Tambahkan middleware catch all 404

```

app.use((req, res) => {
  res.status(404).json({
    message: "Endpoint tidak ditemukan"
  });
});

```

20. Buka Vercel, kemudian login akun pilih new project pilih folder github tugas1-restful-2428240091 lalu scroll kebawah dan klik deploy. Tunggu sampai selesai, kemudian jalankan linknya

21. Buka Postman salin link deploy tadi dan tambahkan endpoint seperti tabel dibawah

//CONTOH

Pilih GET pada postman kemudian pada URL <https://tugas1-restful-2428240091.vercel.app/data-plans>

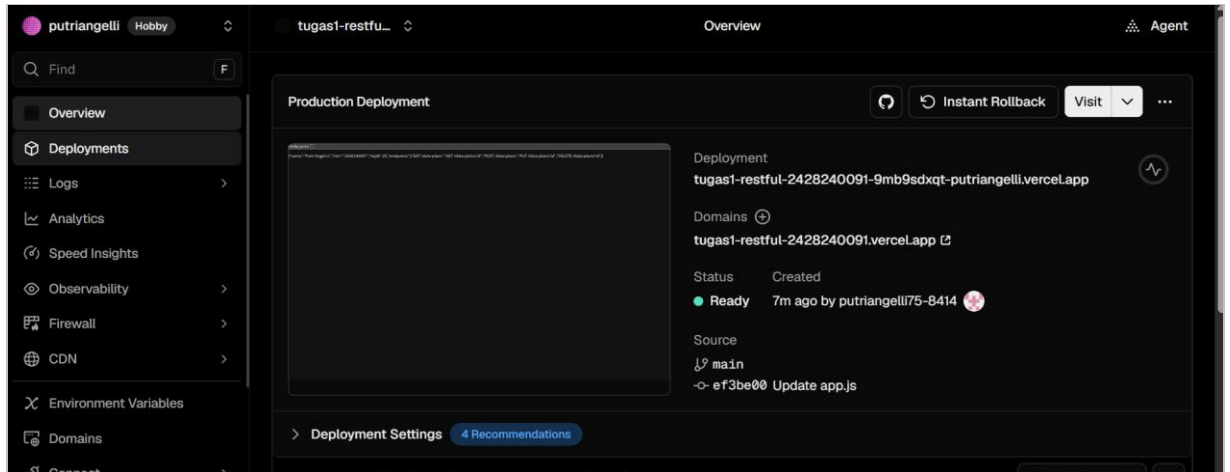
Setelah di cek 1 per 1, Hasilnya berjalan dengan baik, Pengecekan semua endpoint akan di screenshot pada hasil dan pembahasan :

No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/data-plans	—	200	200	Sesuai
2	GET	/data-plans/1	—	200	200	Sesuai
3	GET	/data-plans/99	—	404	404	Sesuai

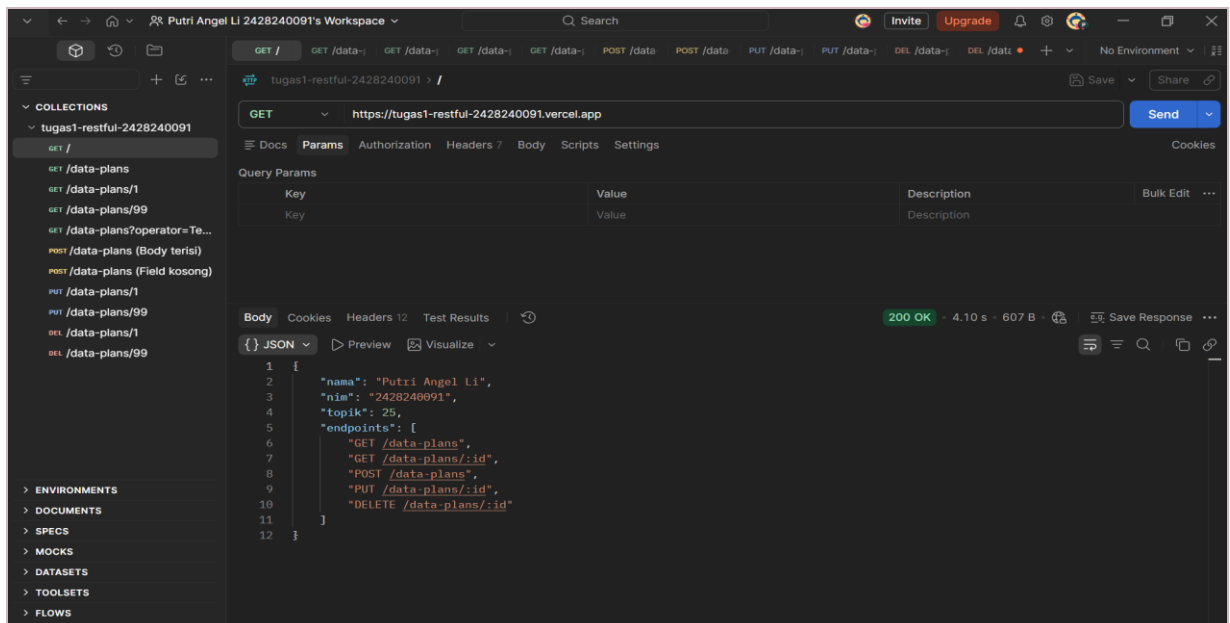
No	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
4	GET	/data-plans?operator=Telkomsel	—	200	200	Sesuai
5	POST	/data-plans	body lengkap	201	201	Sesuai
6	POST	/data-plans	field wajib kosong	400	400	Sesuai
7	PUT	/data-plans/1	body lengkap	200	200	Sesuai
8	PUT	/data-plans/99	body lengkap	404	404	Sesuai
9	DELETE	/data-plans/1	—	200	200	Sesuai
10	DELETE	/data-plans/99	—	404	404	Sesuai

V. Hasil dan Pembahasan

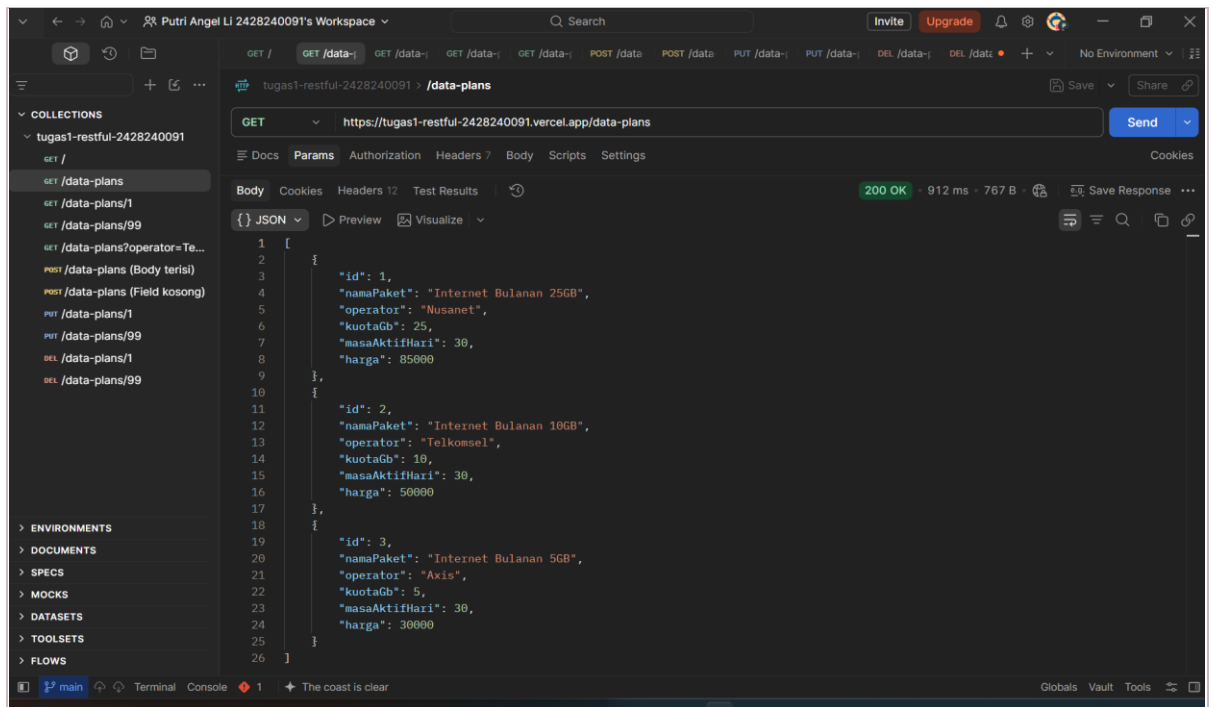
Sertakan tangkapan layar (screenshot) hasil praktikum, disertai penjelasan/pembahasan pada setiap hasil.



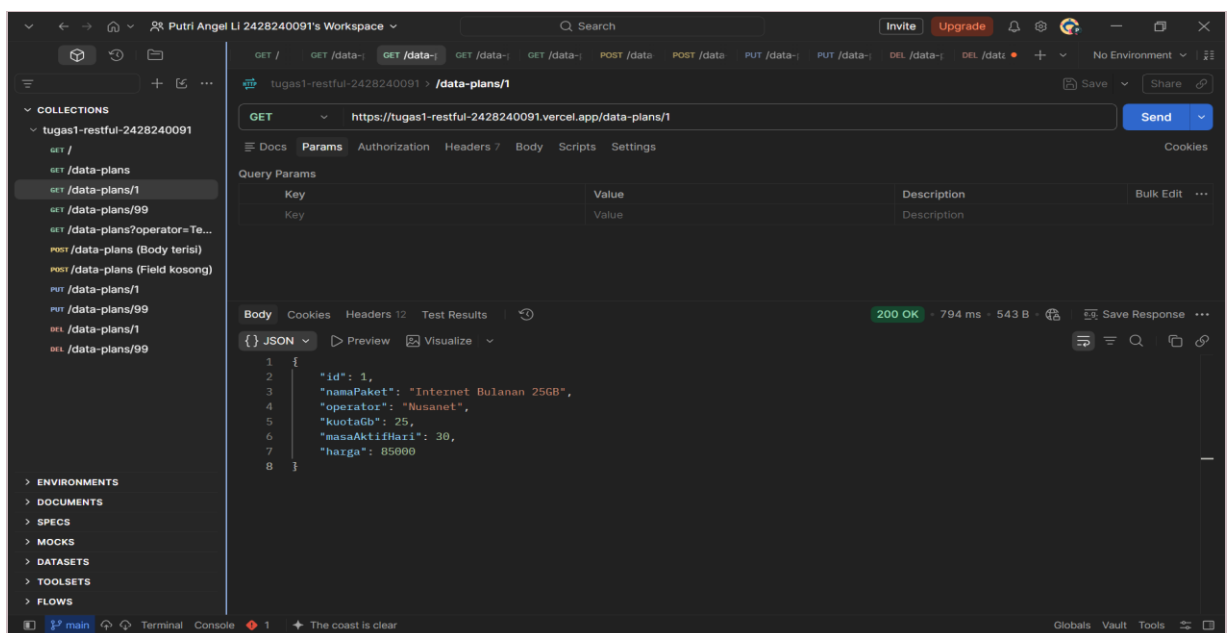
Gambar 1. Hasil Deploy pada Vercel (tugas1-restful-2428240091.vercel.app)



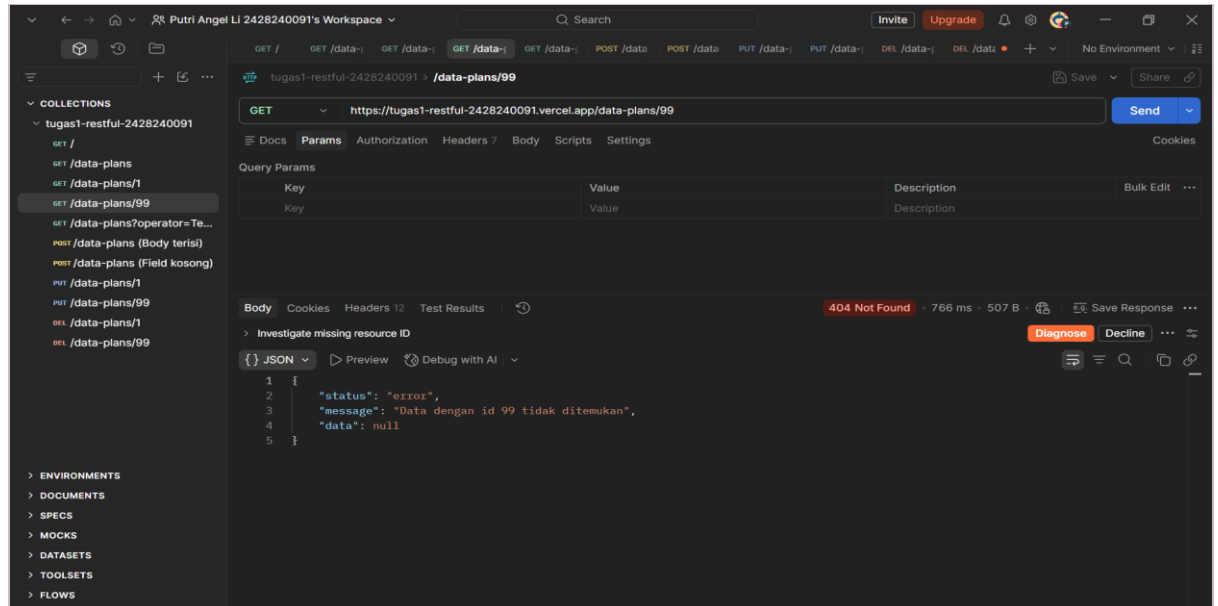
Gambar 2. GET/ <https://tugas1-restful-2428240091.vercel.app> (Menampilkan halaman awal)



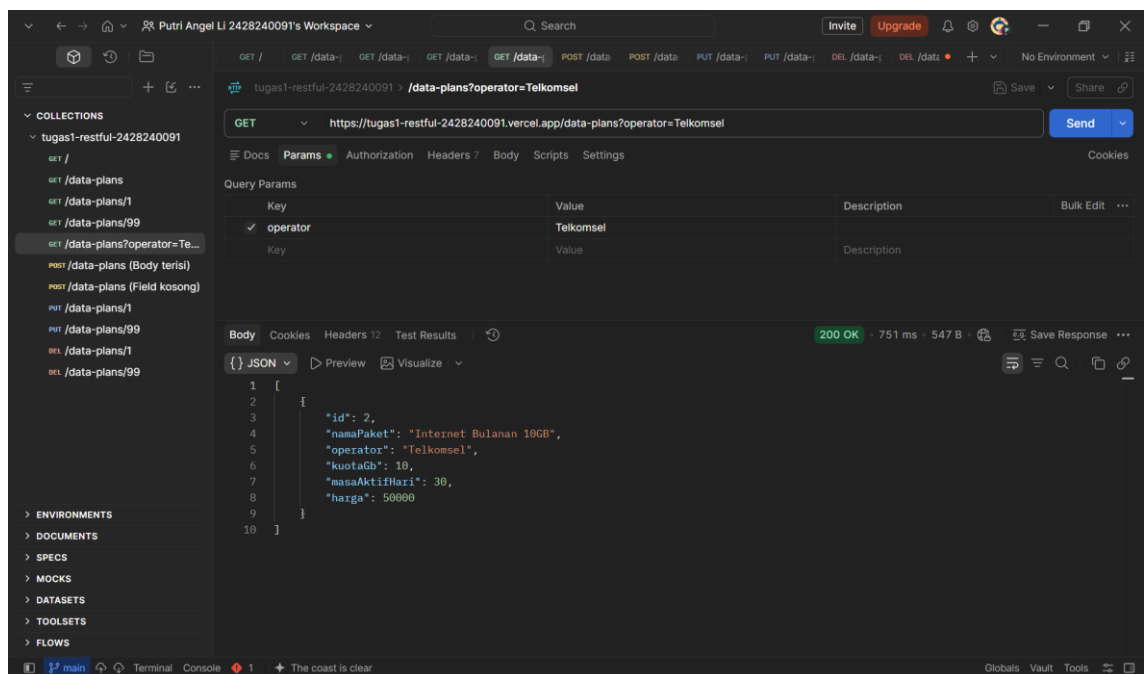
Gambar 3. GET/ <https://tugas1-restful-2428240091.vercel.app/data-plans>
(Menampilkan seluruh data)



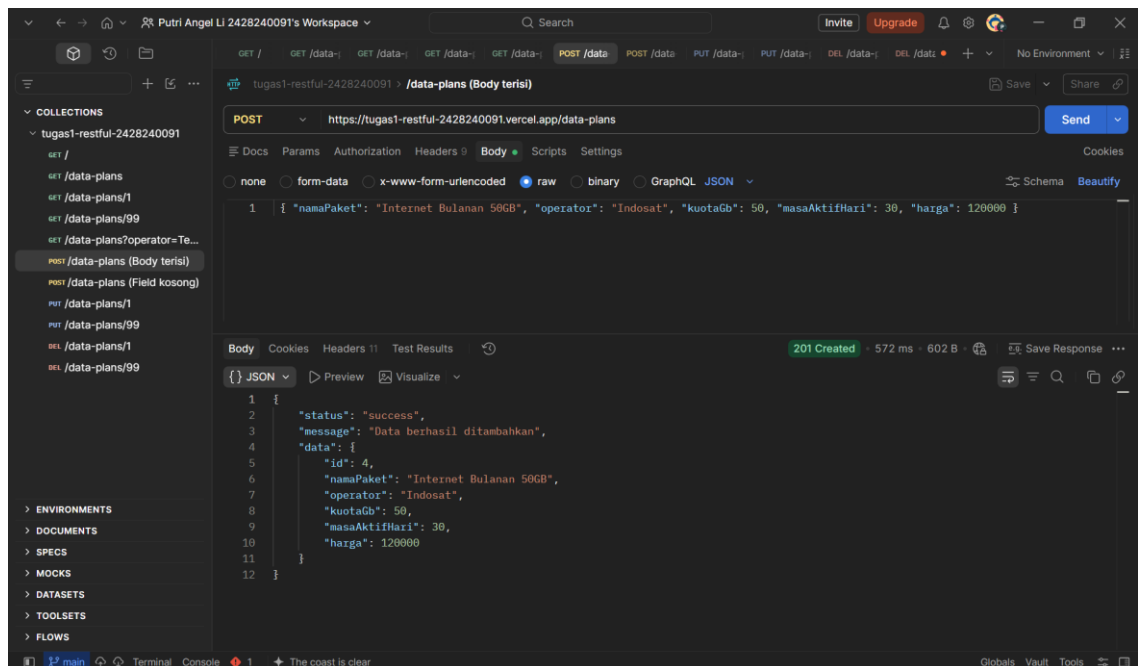
Gambar 4. GET/ <https://tugas1-restful-2428240091.vercel.app/data-plans/1>
(Menampilkan satu data untuk id 1)



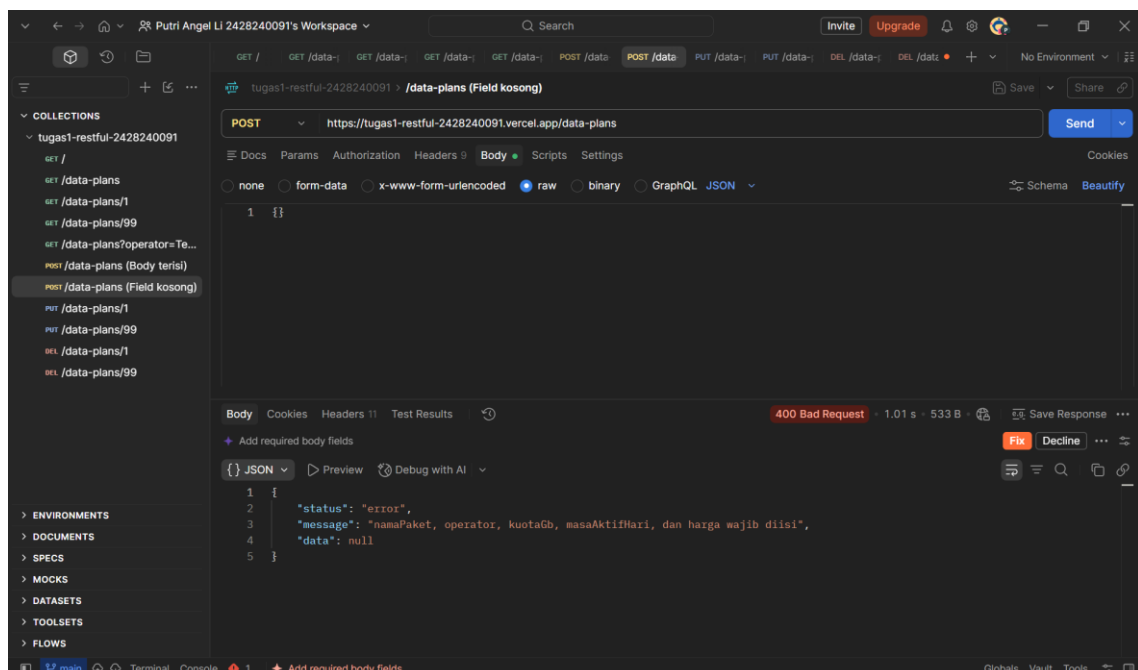
Gambar 5. GET/ <https://tugas1-restful-2428240091.vercel.app/data-plans/99>
(Menampilkan satu data untuk id 99)



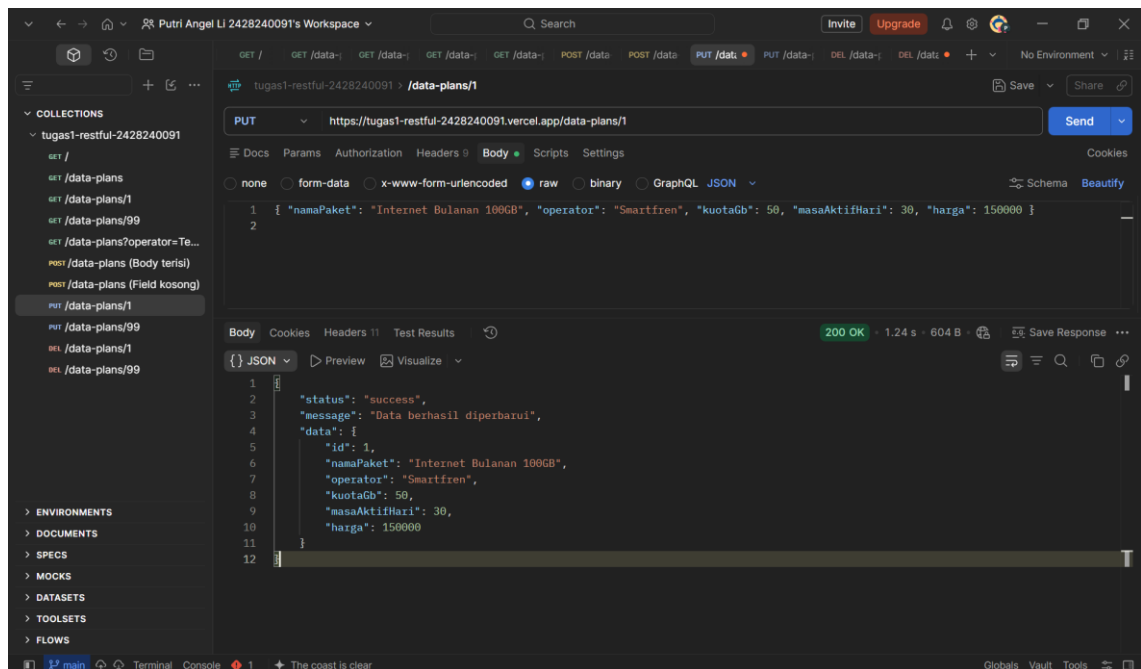
Gambar 6. GET/ <https://tugas1-restful-2428240091.vercel.app/data-plans?operator=Telkomsel> (Menampilkan data operator yang ingin difilter)



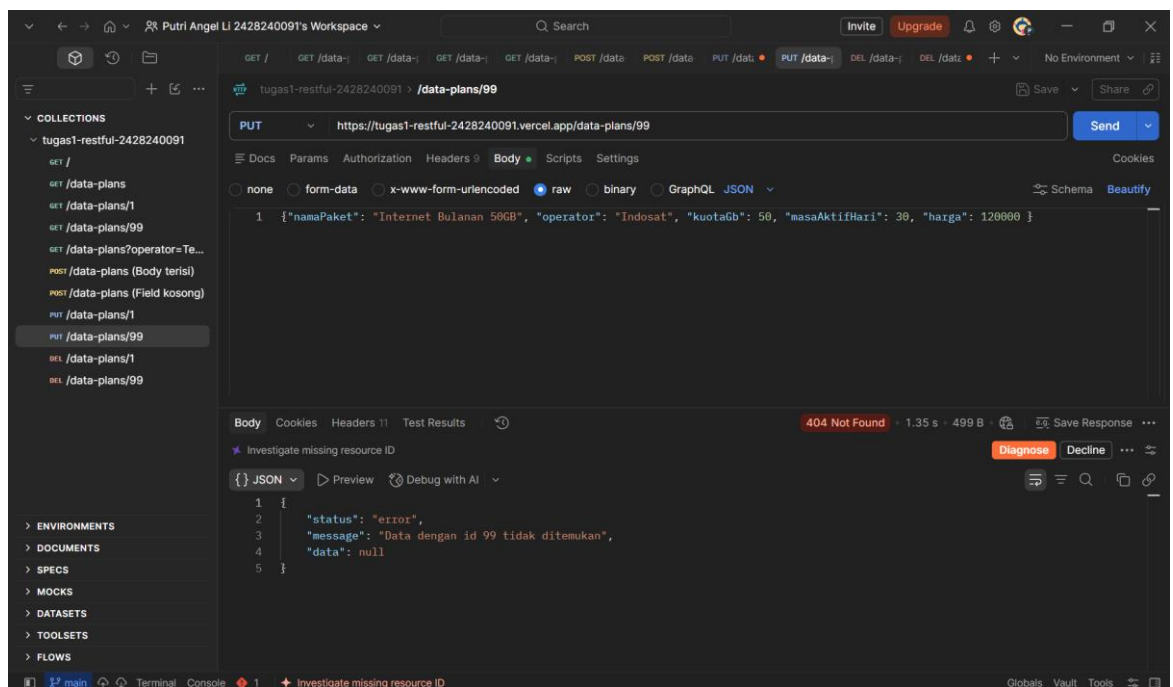
Gambar 7. POST/ <https://tugas1-restful-2428240091.vercel.app/data-plans>
(Menambah data baru dengan body terisi)



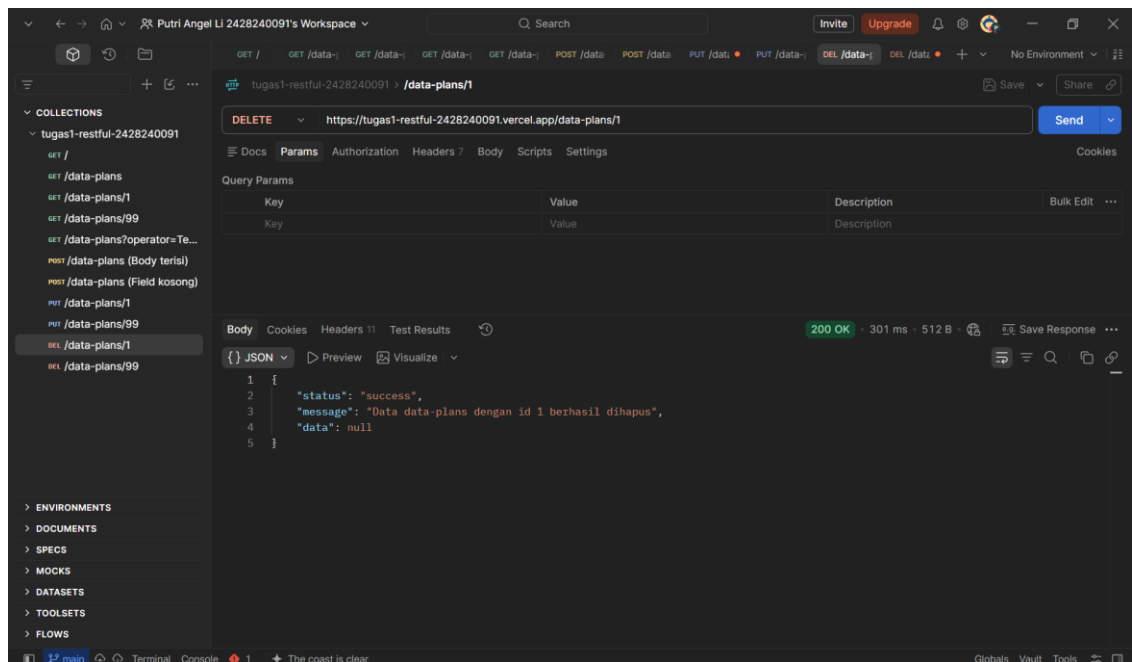
Gambar 8. POST/ <https://tugas1-restful-2428240091.vercel.app/data-plans>
(Menambah data baru dengan field kosong)



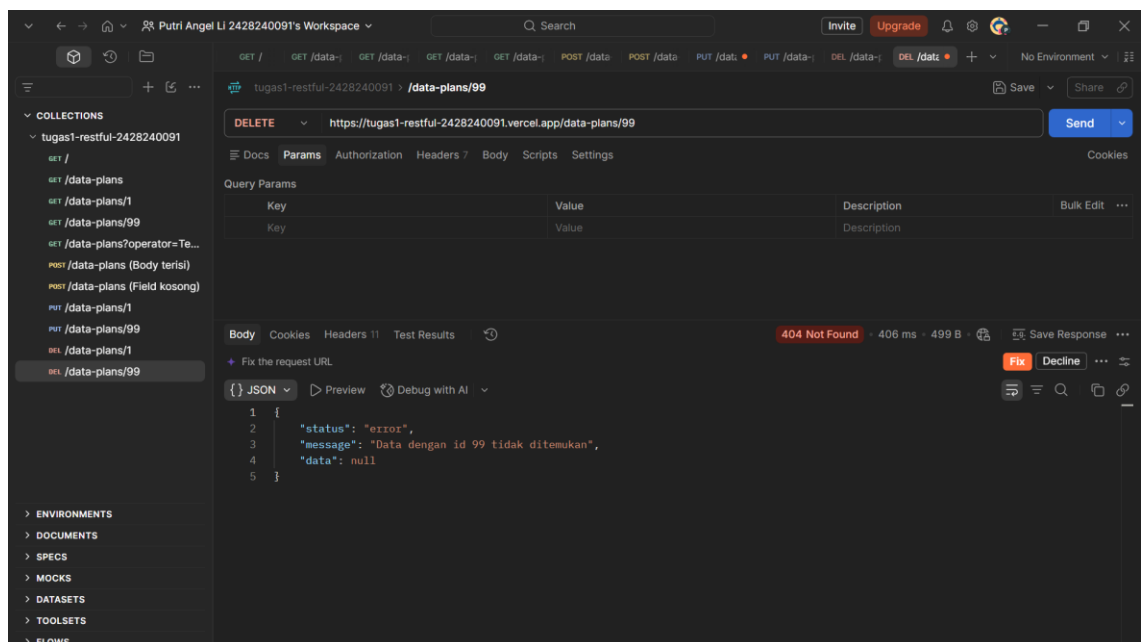
Gambar 9. PUT/ <https://tugas1-restful-2428240091.vercel.app/data-plans/1>
(Mengubah data pada id 1 secara keseluruhan)



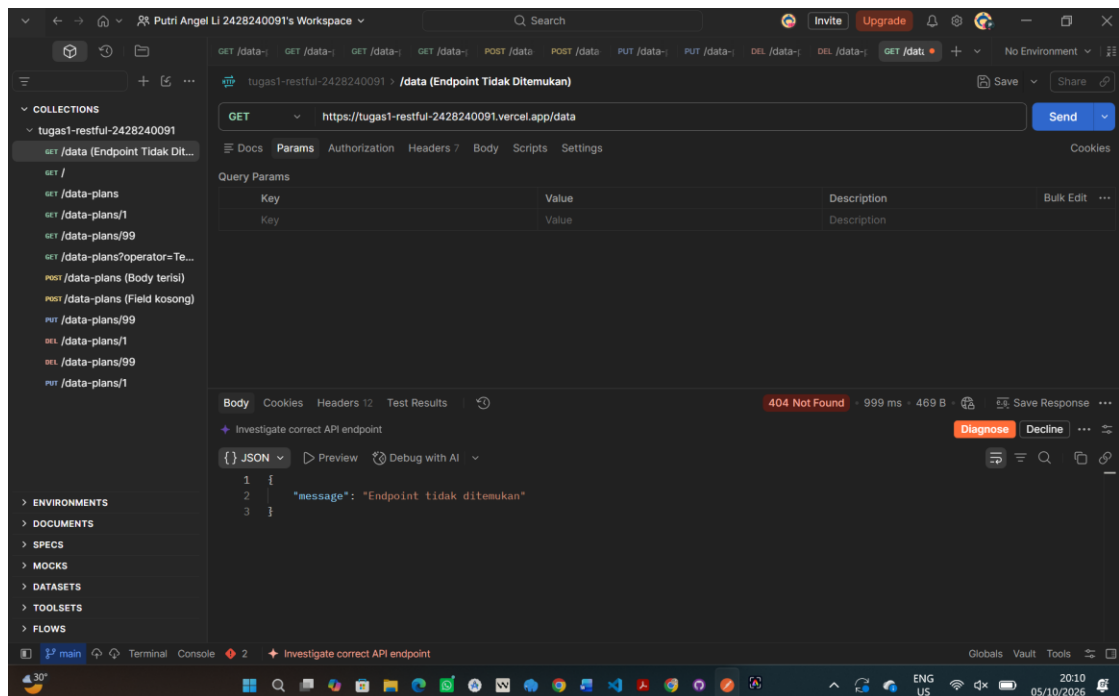
Gambar 10. PUT/ <https://tugas1-restful-2428240091.vercel.app/data-plans/99>
(Mengubah data pada id 99)



Gambar 11. DELETE/ <https://tugas1-restful-2428240091.vercel.app/data-plans/1>
(Menghapus data pada id 1)



Gambar 12. DELETE/ <https://tugas1-restful-2428240091.vercel.app/data-plans/99> (Menghapus data pada id 99)



Gambar 13. GET/ data (Endpoint tidak ditemukan)

Pembahasan:

Praktikum ini menggunakan REST API dengan resource `/data-plans` untuk mengelola data paket telekomunikasi. Setiap method HTTP digunakan sesuai dengan fungsi operasinya terhadap resource. GET digunakan untuk mengambil data, POST digunakan untuk menambahkan data baru, PUT digunakan untuk memperbarui data, sedangkan DELETE digunakan untuk menghapus data. Penggunaan method tersebut mengikuti konsep REST sehingga setiap endpoint memiliki fungsi yang jelas.

1. Pengujian GET

Method GET digunakan untuk mengambil data paket data yang tersimpan pada API. Pengujian dilakukan melalui endpoint:

GET `/data-plans`

Hasil pengujian menunjukkan bahwa endpoint dapat mengembalikan seluruh data paket data dengan status 200 OK. Response diberikan dalam format JSON yang berisi field `id`, `namaPaket`, `operator`, `kuotaGb`, `masaAktifHari`, dan `harga`.

Penggunaan status 200 OK menunjukkan bahwa permintaan berhasil diproses oleh server.

Selain mengambil seluruh data, GET juga digunakan untuk mengambil data berdasarkan ID melalui:

GET /data-plans/1

Endpoint tersebut digunakan ketika client hanya membutuhkan satu data tertentu. Berdasarkan hasil pengujian, data dengan ID 1 dapat ditampilkan dan server memberikan status 200 OK. Pengujian ini telah didokumentasikan pada Gambar 4.

GET juga mendukung filter berdasarkan operator menggunakan query string:

GET /data-plans?operator=Nusanet

Filter tersebut digunakan agar API hanya menampilkan paket dengan operator Nusanet, sehingga client tidak perlu mengambil seluruh data kemudian melakukan penyaringan sendiri.

2. Pengujian POST

Method POST digunakan untuk menambahkan paket data baru ke dalam resource /data-plans. Data dikirim melalui request body dalam format JSON, misalnya:

```
{  
  "namaPaket": "Internet Bulanan 25GB",  
  "operator": "Nusanet",  
  "kuotaGb": 25,  
  "masaAktifHari": 30,  
  "harga": 85000  
}
```

POST dipilih karena operasi yang dilakukan adalah membuat resource atau data baru. Jika data berhasil ditambahkan, server memberikan status 201 Created. Status tersebut menunjukkan bahwa request berhasil dan resource baru telah dibuat.

3. Pengujian PUT

Method PUT digunakan untuk memperbarui data paket yang sudah tersedia. Contohnya:

PUT /data-plans/1

Data perubahan dikirim melalui request body dalam format JSON. PUT digunakan karena client ingin memperbarui data berdasarkan ID tertentu. Jika proses pembaruan berhasil, server memberikan status 200 OK.

Contoh request:

```
{  
  "namaPaket": "Internet Bulanan 30GB",  
  "operator": "Nusanet",  
  "kuotaGb": 30,  
  "masaAktifHari": 30,  
  "harga": 95000  
}
```

Pengujian PUT perlu memastikan bahwa data dengan ID yang dituju memang tersedia dan perubahan yang dikirim sesuai dengan tipe data yang telah ditentukan.

4. Pengujian DELETE

Method DELETE digunakan untuk menghapus data paket berdasarkan ID, misalnya:

DELETE /data-plans/1

DELETE dipilih karena operasi yang dilakukan adalah menghilangkan resource dari sistem. Jika data berhasil dihapus, server dapat memberikan status 200 OK dengan pesan keberhasilan atau 204 No Content apabila server tidak mengirimkan isi response.

Setelah proses DELETE dilakukan, data yang telah dihapus seharusnya tidak lagi muncul ketika dilakukan GET berdasarkan ID tersebut.

5. Pengujian Filter

Pengujian filter dilakukan menggunakan query string:

GET /data-plans?operator=Nusanet

Parameter operator digunakan untuk menyaring data berdasarkan nama operator. Dengan adanya filter ini, client dapat memperoleh data yang lebih spesifik tanpa harus mengambil seluruh data dari server. Jika request berhasil, server memberikan status 200 OK.

6. Skenario Gagal 404

Pengujian 404 dilakukan untuk memastikan API dapat menangani resource atau endpoint yang tidak ditemukan. Salah satu pengujian yang dilakukan adalah:

GET /data-plans/99

Jika ID 99 tidak terdapat dalam database, server memberikan status 404 Not Found dengan pesan bahwa data dengan ID tersebut tidak ditemukan. Pengujian ini telah ditampilkan pada Gambar 5.

Perlu dibedakan bahwa /data-plans/99 merupakan route yang terdaftar, tetapi data dengan ID 99 tidak tersedia. Selain itu, middleware catch-all juga dapat menguji endpoint yang benar-benar tidak terdaftar, misalnya:

GET /data

yang menghasilkan:

```
{  
  
  "message": "Endpoint tidak ditemukan"  
  
}
```

dengan status 404 Not Found.

7. Skenario Gagal 400

Pengujian 400 Bad Request digunakan untuk memastikan server dapat menolak request yang memiliki data tidak valid atau tidak lengkap. Contohnya, client mengirim request POST tanpa field wajib yang menghasilkan:

```
{  
  
  "status": "error",
```

```

    "message": "namaPaket, operator, kuotaGb, masaAktifHari, dan harga
wajib diisi",

    "data": null

}

```

Dengan status 400 bad request

VI. Kendala dan Solusi

Opsional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1		
2		

VII. Kesimpulan

Simpulkan hasil praktikum dikaitkan dengan tujuan pada bagian I.

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa project Express.js berhasil dibuat dan digunakan untuk membangun RESTful API. Routing RESTful berhasil diterapkan untuk menangani berbagai endpoint, serta sistem mampu menerima request dari client dan memberikan response dalam format JSON. Dengan demikian, tujuan praktikum untuk memahami dan mengimplementasikan RESTful API yang mencakup routing, request, dan response JSON telah tercapai.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

MDN Web Docs. *HTTP request methods*. Mozilla Developer Network.

MDN Web Docs. *HTTP response status codes*. Mozilla Developer Network.

JSON. *Introducing JSON*. <https://www.json.org/>

Express.js. *Express - Node.js web application framework*. <https://expressjs.com/>

Lampiran

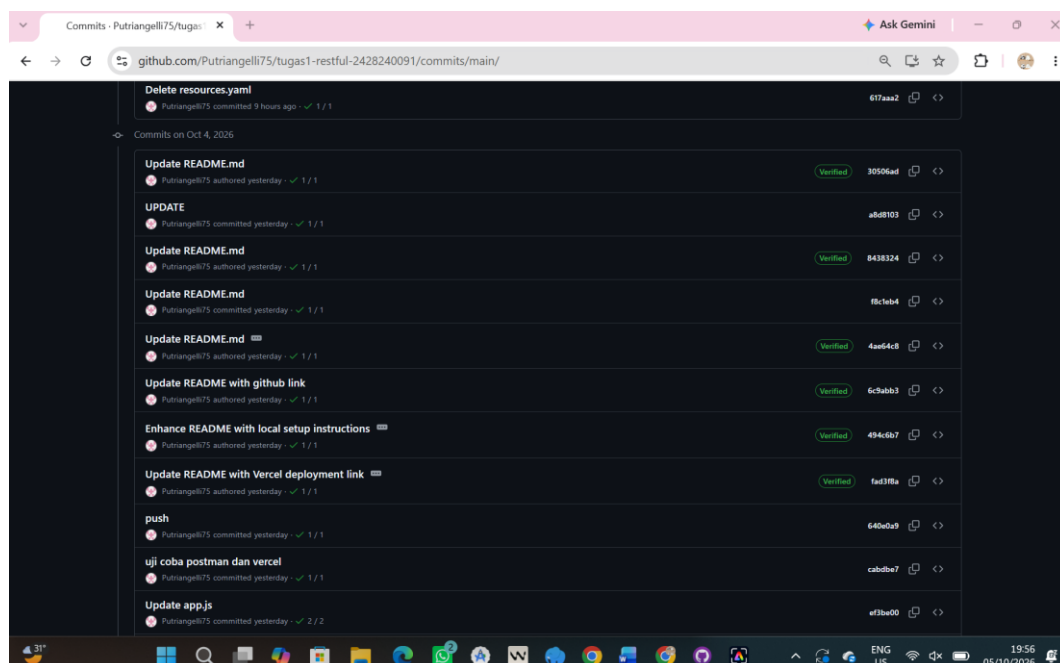
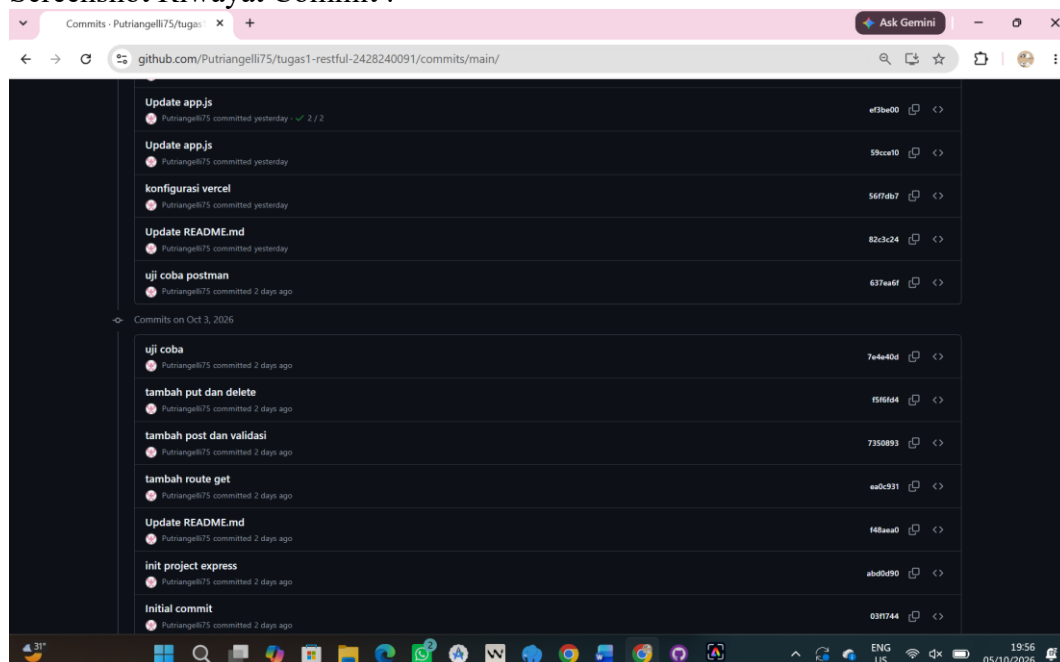
Tautan Repository GitHub:

<https://github.com/Putriangelli75/tugas1-restful-2428240091>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240091.vercel.app/>

Screenshot Riwayat Commit :



Commits on Oct 5, 2026		
Merge branch 'main' of https://github.com/Putriangelli75/tugas1-restful-2428240091	b37f539	<>
Putriangelli75 committed 1 minute ago · ✓ 1 / 1		
update	18ac9c8	<>
Putriangelli75 committed 2 minutes ago		
Update README.mmd	35f075d	<>
Putriangelli75 authored 44 minutes ago · ✓ 1 / 1	Verified	
Update README.mmd	5006dda	<>
Putriangelli75 authored 1 hour ago · ✓ 1 / 1	Verified	
Update topic name in README	a61e63d	<>
Putriangelli75 authored 1 hour ago · ✓ 1 / 1	Verified	
Update README with field and data type information	1383f27	<>
Putriangelli75 authored 1 hour ago · ✓ 1 / 1	Verified	
Create resources.yaml	08afab5	<>
Putriangelli75 committed 1 hour ago · ✓ 1 / 1		
update	8d0b895	<>
Putriangelli75 committed 1 hour ago · ✓ 1 / 1		
Update app.js	e37c946	<>
Putriangelli75 committed 1 hour ago · ✓ 1 / 1		
Delete resources.yaml	617aa2	<>
Putriangelli75 committed 10 hours ago · ✓ 1 / 1		